

Методичка 6. Двумерные списки и игровое поле

1. Зачем нужна эта тема

Обычный список хранит значения в одну линию.

Например:

```
numbers = [10, 20, 30]
```

Но иногда данные удобнее представлять в виде таблицы.

Например:

- игровое поле;
- шахматная доска;
- поле для морского боя;
- карта комнаты;
- расписание;
- таблица оценок;
- экран из символов.

Для таких задач используется двумерный список.

Двумерный список — это список, внутри которого лежат другие списки.

Например, поле 3 на 3:

```
field = [  
    [".", ".", "."],  
    [".", ".", "."],  
    [".", ".", "."]  
]
```

Так можно хранить строки и столбцы.

Эта тема связана с обычными списками и циклами. Чтобы работать с двумерным списком, нужно понимать индексы, `for`, `range` и вложенные циклы.

2. Главная идея темы

Двумерный список похож на таблицу.

У каждого элемента есть два индекса:

- первый индекс — номер строки;
- второй индекс — номер столбца.

```
field[row][col]
```

Если есть поле:

```
field = [  
    ["A", "B", "C"],  
    ["D", "E", "F"],  
    ["G", "H", "I"]  
]
```

Тогда:

- `field[0][0]` — "A" ;
- `field[0][1]` — "B" ;
- `field[1][0]` — "D" ;
- `field[2][2]` — "I" .

Двумерный список — список списков.

Строка — внутренний список.

Столбец — элементы с одинаковым вторым индексом в разных строках.

Координаты — пара чисел, которая показывает положение элемента.

3. Базовый синтаксис

Создание двумерного списка

```
field = [  
    [0, 0, 0],  
    [0, 0, 0],  
    [0, 0, 0]  
]
```

Это таблица 3 на 3.

Получение элемента

```
field = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
print(field[0][0])  
print(field[1][2])
```

Результат:

```
1  
6
```

Изменение элемента

```
field = [  
    [".", ".", "."],  
    [".", ".", "."],  
    [".", ".", "."]  
]  
  
field[1][1] = "X"  
  
print(field)
```

В центре поля появится `"X"` .

Вывод поля

```
field = [  
    [".", ".", "."],  
    [".", ".", "."],  
    [".", ".", "."]  
]  
  
for row in field:  
    print(row)
```

Результат:

```
['.', '.', '.']  
['.', '.', '.']  
['.', '.', '.']
```

Красивый вывод поля

```
field = [  
    [".", ".", "."],  
    [".", ".", "."],  
    [".", ".", "."]  
]  
  
for row in field:  
    for cell in row:  
        print(cell, end=" ")  
    print()
```

Результат:

```
. . .  
. . .  
. . .
```

4. Разбор темы от простого к сложному

Шаг 1. Понять список списков

```
matrix = [  
    [1, 2],  
    [3, 4]  
]  
  
print(matrix)
```

Здесь `matrix` — список, в котором лежат два других списка.

Шаг 2. Получить строку

```
matrix = [  
    [1, 2],  
    [3, 4]  
]  
  
print(matrix[0])  
print(matrix[1])
```

Результат:

```
[1, 2]  
[3, 4]
```

`matrix[0]` — первая строка. `matrix[1]` — вторая строка.

Шаг 3. Получить конкретную ячейку

```
matrix = [  
    [1, 2],  
    [3, 4]  
]  
  
print(matrix[0][1])
```

Результат:

```
2
```

Сначала выбирается строка `matrix[0]`, потом элемент с индексом `1` внутри этой строки.

Шаг 4. Изменить ячейку

```
field = [  
    [".", ".", "."],  
    [".", ".", "."],  
    [".", ".", "."]  
]  
  
field[0][2] = "X"  
  
for row in field:  
    print(row)
```

В первой строке и третьем столбце появится `X`.

Шаг 5. Обойти все элементы

```
field = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
for row in field:  
    for cell in row:  
        print(cell)
```

Здесь используется вложенный цикл:

- внешний цикл перебирает строки;
- внутренний цикл перебирает элементы внутри строки.

Шаг 6. Обойти поле по индексам

```
field = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
for i in range(len(field)):  
    for j in range(len(field[i])):  
        print("Строка:", i, "Столбец:", j, "Значение:", field[i][j])
```

Такой вариант нужен, когда важно знать координаты элемента.

Шаг 7. Создать пустое поле через цикл

```
field = []

for i in range(3):
    row = []

    for j in range(3):
        row.append(".")

    field.append(row)

for row in field:
    print(row)
```

Программа создаёт поле 3 на 3.

Шаг 8. Сделать ход на поле

```
field = [
    [".", ".", "."],
    [".", ".", "."],
    [".", ".", "."]
]

row = int(input("Введите строку от 1 до 3: ")) - 1
col = int(input("Введите столбец от 1 до 3: ")) - 1

field[row][col] = "X"

for line in field:
    for cell in line:
        print(cell, end=" ")
    print()
```

Пользователь вводит координаты от 1 до 3. Программа вычитает 1, потому что индексы в Python начинаются с 0.

5. Частые ошибки учеников

Ошибка 1. Путают строку и столбец

Неправильно:

```
field = [  
    [".", ".", "."],  
    [".", ".", "."],  
    [".", ".", "."]  
]  
  
col = 1  
row = 2  
  
field[col][row] = "X"
```

Почему это ошибка:

Обычно сначала указывается строка, потом столбец: `field[row][col]` .

Правильно:

```
field = [  
    [".", ".", "."],  
    [".", ".", "."],  
    [".", ".", "."]  
]  
  
row = 2  
col = 1  
  
field[row][col] = "X"
```

Ошибка 2. Забывают, что индексы начинаются с 0

Неправильно:

```
field = [  
    [".", ".", "."],  
    [".", ".", "."],  
    [".", ".", "."]  
]  
  
row = int(input("Строка: "))  
col = int(input("Столбец: "))  
  
field[row][col] = "X"
```

Почему это ошибка:

Пользователь обычно вводит строки от 1 до 3, а индексы в Python идут от 0 до 2.

Правильно:

```
field = [  
    [".", ".", "."],  
    [".", ".", "."],  
    [".", ".", "."]  
]  
  
row = int(input("Строка: ")) - 1  
col = int(input("Столбец: ")) - 1  
  
field[row][col] = "X"
```

Ошибка 3. Выходят за границы поля

Неправильно:

```
field = [  
    [".", ".", "."],  
    [".", ".", "."],  
    [".", ".", "."]  
]  
  
field[3][0] = "X"
```

Почему это ошибка:

В поле 3 строки с индексами 0, 1, 2. Индекса 3 нет.

Правильно:

```
field = [  
    [".", ".", "."],  
    [".", ".", "."],  
    [".", ".", "."]  
]  
  
field[2][0] = "X"
```

Ошибка 4. Неправильно создают поле

Неправильно:

```
row = [".", ".", "."]
field = [row, row, row]

field[0][0] = "X"

print(field)
```

Почему это опасно:

Все строки ссылаются на один и тот же список. Если изменить одну строку, изменятся все.

Правильно:

```
field = []

for i in range(3):
    row = []

    for j in range(3):
        row.append(".")

    field.append(row)
```

Ошибка 5. Печатают поле некрасиво

Неправильно:

```
field = [
    [".", ".", "."],
    [".", ".", "."],
    [".", ".", "."]
]

print(field)
```

Почему это неудобно:

Поле выводится в одну строку, и его сложно читать.

Правильно:

```
field = [
    [".", ".", "."],
    [".", ".", "."],
    [".", ".", "."]
]

for row in field:
    for cell in row:
        print(cell, end=" ")
    print()
```

Ошибка 6. Путают перебор строк и ячеек

Неправильно:

```
field = [
    [1, 2, 3],
    [4, 5, 6]
]

for cell in field:
    print(cell)
```

Почему это не ошибка Python, но ошибка понимания:

`cell` здесь на самом деле не ячейка, а целая строка.

Правильно:

```
field = [
    [1, 2, 3],
    [4, 5, 6]
]

for row in field:
    for cell in row:
        print(cell)
```

Ошибка 7. Не проверяют занятость клетки

Неправильно:

```
field[row][col] = "X"
```

Почему это может быть ошибкой:

Если клетка уже занята, ход перезапишет старое значение.

Правильно:

```
if field[row][col] == ".":
    field[row][col] = "X"
else:
    print("Клетка уже занята")
```

6. Мини-примеры для закрепления

Пример 1. Таблица 2 на 2

```
matrix = [
    [1, 2],
    [3, 4]
]

print(matrix[0][0])
print(matrix[1][1])
```

Закрепляется обращение к элементам двумерного списка.

Пример 2. Игровое поле

```
field = [
    [".", ".", "."],
    [".", ".", "."],
    [".", ".", "."]
]

for row in field:
    print(row)
```

Закрепляется хранение поля.

Пример 3. Красивый вывод поля

```

field = [
    ["X", ".", "."],
    [".", "O", "."],
    [".", ".", "X"]
]

for row in field:
    for cell in row:
        print(cell, end=" ")
    print()

```

Закрепляется вложенный цикл.

Пример 4. Изменение клетки

```

field = [
    [".", ".", "."],
    [".", ".", "."],
    [".", ".", "."]
]

field[1][1] = "X"

for row in field:
    print(row)

```

Закрепляется изменение значения по двум индексам.

Пример 5. Подсчёт элементов

```

field = [
    ["X", ".", "."],
    [".", "X", "."],
    [".", ".", "X"]
]

count = 0

for row in field:
    for cell in row:
        if cell == "X":
            count = count + 1

print("Количество X:", count)

```

Закрепляется обход всех ячеек.

Пример 6. Создание поля через цикл

```
field = []

for i in range(3):
    row = []

    for j in range(3):
        row.append(".")

    field.append(row)

print(field)
```

Закрепляется создание двумерного списка.

Пример 7. Проверка клетки

```
field = [
    [".", ".", "."],
    [".", "X", "."],
    [".", ".", "."]
]

row = 1
col = 1

if field[row][col] == ".":
    print("Клетка свободна")
else:
    print("Клетка занята")
```

Закрепляется проверка значения в ячейке.

7. Практические задания

Лёгкий уровень

1. Создать поле 3 на 3 из точек и вывести его на экран.
2. Создать таблицу 2 на 2 с числами и вывести каждый элемент.
3. Дан двумерный список 3 на 3. Вывести элемент в центре.
4. В поле 3 на 3 поставить "X" в левый верхний угол.
5. Вывести игровое поле красиво: без квадратных скобок и кавычек.

Средний уровень

1. Создать поле 5 на 5 из символов `"."`.
2. Пользователь вводит строку и столбец от 1 до 3. Поставить `"X"` в выбранную клетку.
3. Дан двумерный список чисел. Посчитать сумму всех элементов.
4. Дан двумерный список символов. Посчитать количество символов `"X"`.
5. Сделать проверку: если выбранная клетка занята, вывести сообщение.

Повышенный уровень

1. Создать поле 5 на 5 и поставить `"X"` на главную диагональ.
2. Создать поле 5 на 5 и поставить `"X"` по рамке поля.
3. Реализовать маленькую программу, где пользователь несколько раз вводит координаты, а программа ставит `"X"` на поле, если клетка свободна.

8. Контрольные вопросы

1. Что такое двумерный список?
2. Чем двумерный список отличается от обычного списка?
3. Что означает запись `field[row][col]`?
4. Какой индекс отвечает за строку?
5. Какой индекс отвечает за столбец?
6. Почему пользовательский ввод координат часто нужно уменьшать на 1?
7. Что такое вложенный цикл?
8. Как красиво вывести игровое поле?
9. Почему нельзя выходить за границы списка?
10. Почему опасно создавать поле через повторение одной и той же строки?
11. Как проверить, свободна ли клетка?
12. Где в реальных программах используются двумерные списки?

9. Краткий конспект в конце

Главная идея темы: двумерный список хранит данные в виде таблицы.

Основные конструкции:

```
field = [  
    [".", ".", "."],  
    [".", ".", "."],  
    [".", ".", "."]  
]
```

```
field[0][0] = "X"
```

```
print(field[row][col])
```

```
for row in field:  
    for cell in row:  
        print(cell, end=" ")  
    print()
```

Важно запомнить:

- двумерный список — это список списков;
- первый индекс — строка;
- второй индекс — столбец;
- индексы начинаются с 0;
- для обхода всех ячеек часто используется вложенный цикл;
- игровое поле удобно хранить как двумерный список.

Частые ошибки:

- путать строку и столбец;
- забывать вычитать 1 из пользовательских координат;
- выходить за границы поля;
- неправильно создавать одинаковые строки;
- не проверять, свободна ли клетка.

10. Что ученик должен уметь после методички

После этой темы ученик должен уметь:

- создавать двумерный список;
- получать элемент по двум индексам;
- изменять ячейку поля;
- выводить двумерный список на экран;
- красиво выводить игровое поле;
- использовать вложенные циклы;

- создавать поле через цикл;
- проверять значение в клетке;
- работать с координатами строки и столбца;
- применять двумерные списки для простых игровых задач.